RNA-seq analysis in R

(index.html)

Alignment and feature counting

Authors: Belinda Phipson, Maria Doyle, Harriet Dashnow

This material has been created using the following sources:
http://www.statsci.org/smyth/pubs/QLedgeRPreprint.pdf (Lun, Chen, and Smyth 2016) http://bioinf.wehi.edu.au/RNAseqCaseStudy/

Packages used:
Rsubread

Data files needed:
Mouse chromosome 1 Rsubread index files (~400MB).
Targets2.txt
The 12 fastq.gz files for the mouse dataset.

Mouse mammary data (fastq files): https://figshare.com/s/f5d63d8c265a05618137 (https://figshare.com/s/f5d63d8c265a05618137) You should download these files and place them in your `/data` directory.

GEO entry for the dataset:
http://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE60450

The raw reads were downloaded from SRA from the link given in GEO for the dataset (ftp://ftp-trace.ncbi.nlm.nih.gov/sra/sra-instant/reads/ByStudy/sra/SRP%2FSRP045%2FSRP045534). These files are in .sra format. The sra toolkit from NCBI was used to convert the .sra files to .fastq files using the fastq-dump command.

Downloading genome files

We have provided the index files for chromosome 1 for the mouse genome build mm10 for this workshop in order to save time on building the index. However, full genome fasta files for a number of different genomes are available to download from the UCSC genome browser, see http://hgdownload.soe.ucsc.edu/downloads.html; from NCBI: http://www.ncbi.nlm.nih.gov/genome; or from ENSEMBL: http://asia.ensembl.org/info/data/ftp/index.html.

Introduction and data import

For the purpose of this workshop, we are going to be working with a small part of the mouse reference genome (chromosome 1) to demonstrate how to do read alignment and counting using R. Mapping reads to the genome is a very important task, and many different aligners are available, such as bowtie (Langmead and Salzberg 2012), topHat (Trapnell, Pachter, and Salzberg 2009), STAR (Dobin et al. 2013) and Rsubread (Liao, Smyth, and Shi 2013). Rsubread is the only aligner that can run in R. Most alignment tools are run in a linux environment, and they are very computationally intensive. Most mapping tasks require larger computers than an average laptop, so usually read mapping is done on a server in a linux-like environment. Here we are only going to be mapping 1000 reads from each sample from our mouse lactation dataset (Fu et al. 2015), and we will only be mapping to chromosome 1. This is so that everyone can have a go at alignment and counting on their laptops using RStudio.

Don't forget to open a new project specifying the directory where you have saved all the data files for day 2

First, let's load the Rsubread package into R.

```
library(Rsubread)
```

Earlier we put all the sequencing read data (.fastq.gz files) in the data directory. Now we need to find them in order to tell the Rsubread aligner which files to look at. We can search for all .fastq.gz files in the data directory using the `list.files` command. The pattern argument takes a regular expression. In this case we are using the `$` to mean the end of the string, so we will only get files that end in ".fastq.gz"

```
fastq.files <- list.files(path = "./data", pattern = ".fastq.gz$", full.names = TRUE)
fastq.files
```

```
 [1] "./data/SRR1552444.fastq.gz" "./data/SRR1552445.fastq.gz"
 [3] "./data/SRR1552446.fastq.gz" "./data/SRR1552447.fastq.gz"
 [5] "./data/SRR1552448.fastq.gz" "./data/SRR1552449.fastq.gz"
 [7] "./data/SRR1552450.fastq.gz" "./data/SRR1552451.fastq.gz"
 [9] "./data/SRR1552452.fastq.gz" "./data/SRR1552453.fastq.gz"
[11] "./data/SRR1552454.fastq.gz" "./data/SRR1552455.fastq.gz"
```

Alignment

Build the index

Read sequences are stored in compressed (gzipped) FASTQ files. Before the differential expression analysis can proceed, these reads must be aligned to the mouse genome and counted into annotated genes. This can be achieved with functions in the Rsubread package.

The first step in performing the alignment is to build an index. In order to build an index you need to have the fasta file (.fa), which can be downloaded from the UCSC genome browser. Here we are building the index just for chromosome 1. This may take several minutes to run. Building the full index using the whole mouse genome usually takes about 30 minutes to an hr on a server. We won't be building the index in the workshop due to time constraints, we have provided the index files for you. The command below assumes the chr1 genome information for mm10 is stored in the "chr1.fa" file.

```
# buildindex(basename="chr1_mm10",reference="chr1.fa")
```

The above command will generate the index files in the working directory. In this example, the prefix for the index files is chr1_mm10. You can see the additional files generated using the `dir` command, which will list every file in your current working directory.

```
dir()
```

Aligning reads to chromosome 1 of reference genome

Now that we have generated our index, we can align our reads using the `align` command. There are often numerous mapping parameters that we can specify, but usually the default mapping parameters for the `align` function are fine. If we had paired end data, we would specify the second read file/s using the `readfile2` argument. Our mouse data comprises 100bp single end reads.

We can specify the output files, or we can let `Rsubread` choose the output file names for us. The default output file name is the filename with ".subread.BAM" added at the end.

Now we can align our 12 fastq.gz files using the `align` command.

```
align(index="data/chr1_mm10",readfile1=fastq.files)
```

This will align each of the 12 samples one after the other. As we're only using a subset of 1000 reads per sample, aligning should just take a minute or so for each sample. To run the full samples from this dataset would take several hours per sample. The BAM files are saved in the working directory.

To see how many parameters you can change try the `args` function:

```
args(align)
```

```
function (index, readfile1, readfile2 = NULL, type = "rna", input_format = "gzFASTQ",
    output_format = "BAM", output_file = paste(as.character(readfile1),
        "subread", output_format, sep = "."), nsubreads = 10,
    TH1 = 3, TH2 = 1, maxMismatches = 3, nthreads = 1, indels = 5,
    complexIndels = FALSE, phredOffset = 33, unique = TRUE, nBestLocations = 1,
    minFragLength = 50, maxFragLength = 600, PE_orientation = "fr",
    nTrim5 = 0, nTrim3 = 0, readGroupID = NULL, readGroup = NULL,
    color2base = FALSE, DP_GapOpenPenalty = -1, DP_GapExtPenalty = 0,
    DP_MismatchPenalty = 0, DP_MatchScore = 2, detectSV = FALSE)
NULL
```

In this example we have kept many of the default settings, which have been optimised to work well under a variety of situations. The default setting for `align` is that it only keeps reads that uniquely map to the reference genome. For testing differential expression of genes, this is what we want, as the reads are unambigously assigned to one place in the genome, allowing for easier interpretation of the results. Understanding all the different parameters you can change involves doing a lot of reading about the aligner that you are using, and can take a lot of time to understand! Today we won't be going into the details of the parameters you can change, but you can get more information from looking at the help:

```
?align
```

We can get a summary of the proportion of reads that mapped to the reference genome using the `propmapped` function.

```
bam.files <- list.files(path = "./data", pattern = ".BAM$", full.names = TRUE)
bam.files
```

```
 [1] "./data/SRR1552444.fastq.gz.subread.BAM"
 [2] "./data/SRR1552445.fastq.gz.subread.BAM"
 [3] "./data/SRR1552446.fastq.gz.subread.BAM"
 [4] "./data/SRR1552447.fastq.gz.subread.BAM"
 [5] "./data/SRR1552448.fastq.gz.subread.BAM"
 [6] "./data/SRR1552449.fastq.gz.subread.BAM"
 [7] "./data/SRR1552450.fastq.gz.subread.BAM"
 [8] "./data/SRR1552451.fastq.gz.subread.BAM"
 [9] "./data/SRR1552452.fastq.gz.subread.BAM"
[10] "./data/SRR1552453.fastq.gz.subread.BAM"
[11] "./data/SRR1552454.fastq.gz.subread.BAM"
[12] "./data/SRR1552455.fastq.gz.subread.BAM"
```

```
props <- propmapped(files=bam.files)
```

```
The input file is opened as a BAM file.
The fragments in the input file are being counted.
Your operation system does not allow a single process to open more then 400 files. You may need to change this setting
by using a 'ulimit -n 500' command, or the program may crash.
Finished. All records: 1000; all fragments: 1000; mapped fragments: 133; the mappability is 13.30%
The input file is opened as a BAM file.
The fragments in the input file are being counted.
Your operation system does not allow a single process to open more then 400 files. You may need to change this setting
by using a 'ulimit -n 500' command, or the program may crash.
Finished. All records: 1000; all fragments: 1000; mapped fragments: 130; the mappability is 13.00%
The input file is opened as a BAM file.
The fragments in the input file are being counted.
Your operation system does not allow a single process to open more then 400 files. You may need to change this setting
by using a 'ulimit -n 500' command, or the program may crash.
Finished. All records: 1000; all fragments: 1000; mapped fragments: 111; the mappability is 11.10%
The input file is opened as a BAM file.
The fragments in the input file are being counted.
Your operation system does not allow a single process to open more then 400 files. You may need to change this setting
by using a 'ulimit -n 500' command, or the program may crash.
Finished. All records: 1000; all fragments: 1000; mapped fragments: 103; the mappability is 10.30%
The input file is opened as a BAM file.
The fragments in the input file are being counted.
Your operation system does not allow a single process to open more then 400 files. You may need to change this setting
by using a 'ulimit -n 500' command, or the program may crash.
Finished. All records: 1000; all fragments: 1000; mapped fragments: 53; the mappability is 5.30%
The input file is opened as a BAM file.
The fragments in the input file are being counted.
Your operation system does not allow a single process to open more then 400 files. You may need to change this setting
by using a 'ulimit -n 500' command, or the program may crash.
Finished. All records: 1000; all fragments: 1000; mapped fragments: 67; the mappability is 6.70%
The input file is opened as a BAM file.
The fragments in the input file are being counted.
Your operation system does not allow a single process to open more then 400 files. You may need to change this setting
by using a 'ulimit -n 500' command, or the program may crash.
Finished. All records: 1000; all fragments: 1000; mapped fragments: 119; the mappability is 11.90%
The input file is opened as a BAM file.
The fragments in the input file are being counted.
Your operation system does not allow a single process to open more then 400 files. You may need to change this setting
by using a 'ulimit -n 500' command, or the program may crash.
Finished. All records: 1000; all fragments: 1000; mapped fragments: 145; the mappability is 14.50%
The input file is opened as a BAM file.
The fragments in the input file are being counted.
Your operation system does not allow a single process to open more then 400 files. You may need to change this setting
by using a 'ulimit -n 500' command, or the program may crash.
Finished. All records: 1000; all fragments: 1000; mapped fragments: 133; the mappability is 13.30%
The input file is opened as a BAM file.
The fragments in the input file are being counted.
Your operation system does not allow a single process to open more then 400 files. You may need to change this setting
by using a 'ulimit -n 500' command, or the program may crash.
Finished. All records: 1000; all fragments: 1000; mapped fragments: 142; the mappability is 14.20%
The input file is opened as a BAM file.
The fragments in the input file are being counted.
Your operation system does not allow a single process to open more then 400 files. You may need to change this setting
by using a 'ulimit -n 500' command, or the program may crash.
Finished. All records: 1000; all fragments: 1000; mapped fragments: 127; the mappability is 12.70%
The input file is opened as a BAM file.
The fragments in the input file are being counted.
Your operation system does not allow a single process to open more then 400 files. You may need to change this setting
by using a 'ulimit -n 500' command, or the program may crash.
Finished. All records: 1000; all fragments: 1000; mapped fragments: 113; the mappability is 11.30%
```

```
props
```

```
                              Samples NumTotal NumMapped PropMapped
1  ./data/SRR1552444.fastq.gz.subread.BAM     1000       133      0.133
2  ./data/SRR1552445.fastq.gz.subread.BAM     1000       130      0.130
3  ./data/SRR1552446.fastq.gz.subread.BAM     1000       111      0.111
4  ./data/SRR1552447.fastq.gz.subread.BAM     1000       103      0.103
5  ./data/SRR1552448.fastq.gz.subread.BAM     1000        53      0.053
6  ./data/SRR1552449.fastq.gz.subread.BAM     1000        67      0.067
7  ./data/SRR1552450.fastq.gz.subread.BAM     1000       119      0.119
8  ./data/SRR1552451.fastq.gz.subread.BAM     1000       145      0.145
9  ./data/SRR1552452.fastq.gz.subread.BAM     1000       133      0.133
10 ./data/SRR1552453.fastq.gz.subread.BAM     1000       142      0.142
11 ./data/SRR1552454.fastq.gz.subread.BAM     1000       127      0.127
12 ./data/SRR1552455.fastq.gz.subread.BAM     1000       113      0.113
```

Challenge

1. Try aligning the fastq files allowing multi-mapping reads (set `unique = FALSE`), and allowing for up to 6 “best” locations to be reported (`nBestLocations = 6`). Specify the output file names (bam.files.multi) by substituting “.fastq.gz” with “.multi.bam” so we don’t overwrite our unique alignment bam files.
2. Look at the proportion of reads mapped and see if we get any more reads mapping by specifying a less stringent criteria.

Quality control

We can have a look at the quality scores associated with each base that has been called by the sequencing machine using the `qualityScores` function in *Rsubread*.

Let’s first extract quality scores for 100 reads for the file “SRR1552450.fastq.gz”.

```
# Extract quality scores
qs <- qualityScores(filename="data/SRR1552450.fastq.gz",nreads=100)
```

```
qualityScores Rsubread 1.22.3

Scan the input file...
Totally 1000 reads were scanned; the sampling interval is 9.
Now extract read quality information...

Completed successfully. Quality scores for 100 reads (equally spaced in the file) are returned.
```

```
# Check dimension of qs
dim(qs)
```

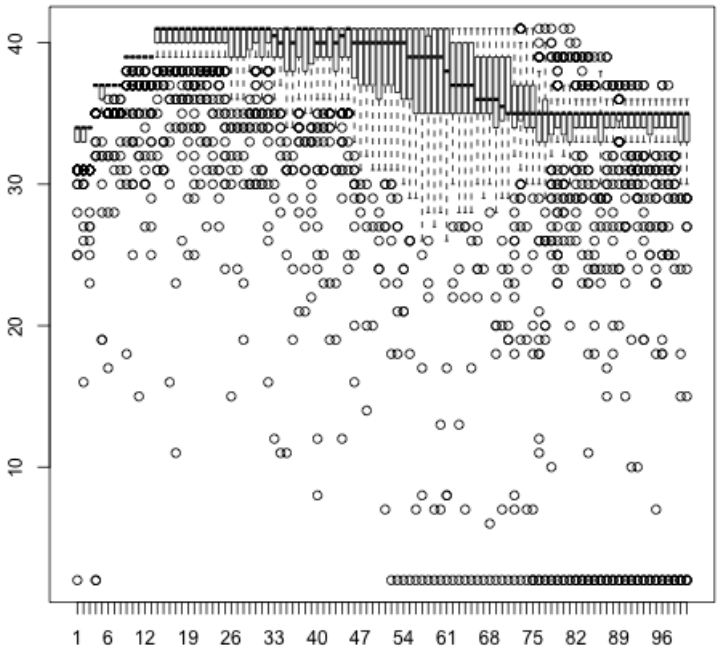
```
[1] 100 100
```

```
# Check first few elements of qs with head
head(qs)
```

```
     1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
[1,] 31 34 31 37 37 37 37 37 39 37 39 39 39 41 40 41 41 41 36 38 40 41 39
[2,] 34 34 34 37 37 37 37 37 39 39 39 39 39 41 41 41 41 39 40 41 41 41 41
[3,] 34 34 31 37 37 37 37 37 39 39 39 39 39 41 41 41 41 41 41 41 41 41 41
[4,] 34 34 34 37 37 37 37 37 39 39 39 39 39 41 41 41 41 39 40 40 41 41 41
[5,] 34 34 34 37 37 37 37 37 39 39 39 39 39 41 41 41 40 41 41 41 41 41 41
[6,] 34 34 34 37 37 37 37 37 39 39 39 39 39 41 41 41 41 41 41 41 41 41 41
     24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46
[1,] 40 38 40 40 40 40 40 38 40 40 40 38 37 39 40 40 41 40 41 41 38 40 40
[2,] 41 41 41 41 41 41 41 41 41 38 40 41 41 41 41 41 41 41 41 41 41 41 41
[3,] 41 41 41 41 41 41 40 40 41 38 38 38 40 41 40 41 39 40 40 41 41 41 39
[4,] 41 40 41 41 41 41 41 40 41 41 41 41 40 41 41 41 41 41 41 41 41 41 41
[5,] 40 41 41 41 41 41 39 40 41 41 41 41 40 41 41 41 41 41 41 41 41 41 41
[6,] 41 41 36 40 40 41 41 41 41 40 38 40 38 40 41 41 40 41 41 41 40 40 41
     47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69
[1,] 40 41 41 41 40 40 36 40 38 37 37 35 35 33 35 35 33 35 34 35 35 35 34
[2,] 41 41 41 41 41 41 41 41 41 41 41 40 41 41 41 39 39 39 39 37 37 37 37
[3,] 40 38 40 40 37 39 40 41 41 40 37 39 39 39 37 37 37 37 37 37 36 36 35
[4,] 41 41 41 41 41 41 41 41 41 40 40 41 40 41 41 41 41 40 41 41 39 39 39
[5,] 41 41 41 41 41 40 41 41 41 41 40 41 41 41 38 40 41 41 41 41 39 39 39
[6,] 40 40 40 41 40 39 33 36 35 34 35 35 35 35 35 35 35 33 33 33 34 31 31
     70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92
[1,] 34 34 35 35 35 35 35 35 35 36 35 35 35 34 34 35 34 35 35 35 35 35 35
[2,] 36 36 36 34 36 35 35 35 35 35 35 35 35 35 35 37 35 36 35 35 35 35 35
[3,] 36 35 33 35 35 35 35 34 36 35 35 34 35 35 35 34 35 35 35 35 35 33 35
[4,] 39 37 37 37 37 37 37 36 36 36 36 36 36 35 35 35 35 35 35 35 35 35 34
[5,] 39 39 38 37 37 37 37 37 35 35 35 35 35 36 35 35 35 35 35 35 35 35 35
[6,] 34 29 34 33 34 35 32 32 34 34 33 31 34 34 18 24 32 33 35 35 27 32 31
     93 94 95 96 97 98 99 100
[1,] 35 35 35 35 35 35 35  33
[2,] 35 36 36 35 35 34 34  35
[3,] 35 35 31 34 35 35 33  33
[4,] 35 35 35 35 35 35 35  35
[5,] 35 36 35 35 35 35 35  35
[6,] 32 34 34 35 34 34 33  34
```

A quality score of 30 corresponds to a 1 in 1000 chance of an incorrect base call. (A quality score of 10 is a 1 in 10 chance of an incorrect base call.) To look at the overall distribution of quality scores across the 100 reads, we can look at a boxplot

```
boxplot(qs)
```



Challenge

1. Extract quality scores for SRR1552451.fastq.gz for 50 reads.
2. Plot a boxplot of the quality scores for SRR1552451.fastq.gz.

Counting

Now that we have figured out where each read comes from in the genome, we need to summarise the information across genes or exons. The alignment produces a set of BAM files, where each file contains the read alignments for each library. In the BAM file, there is a chromosomal location for every read that mapped uniquely. The mapped reads can be counted across mouse genes by using the `featureCounts` function. `featureCounts` contains built-in annotation for mouse (mm9, mm10) and human (hg19) genome assemblies (NCBI refseq annotation).

The code below uses the exon intervals defined in the NCBI refseq annotation of the mm10 genome. Reads that map to exons of genes are added together to obtain the count for each gene, with some care taken with reads that span exon-exon boundaries. `featureCounts` takes all the BAM files as input, and outputs an object which includes the count matrix, similar to the count matrix we have been working with on Day 1. Each sample is a separate column, each row is a gene.

```
fc <- featureCounts(bam.files, annot.inbuilt="mm10")
```

```
# See what slots are stored in fc
names(fc)
```

```
[1] "counts"     "annotation" "targets"    "stat"
```

The statistics of the read mapping can be seen with fc$stats. This reports the numbers of unassigned reads and the reasons why they are not assigned (eg. ambiguity, multi-mapping, secondary alignment, mapping quality, fragment length, chimera, read duplicate, non-junction and so on), in addition to the number of successfully assigned reads for each library. (We know the real reason why the majority of the reads aren't mapping - they're not from chr 1!)

```
## Take a look at the featurecounts stats
fc$stat
```

```
                   Status ..data.SRR1552444.fastq.gz.subread.BAM
1                Assigned                                     48
2      Unassigned_Ambiguity                                    0
3   Unassigned_MultiMapping                                    0
4     Unassigned_NoFeatures                                   85
5       Unassigned_Unmapped                                  867
6  Unassigned_MappingQuality                                   0
7  Unassigned_FragmentLength                                   0
8        Unassigned_Chimera                                    0
9      Unassigned_Secondary                                    0
10   Unassigned_Nonjunction                                    0
11     Unassigned_Duplicate                                    0
   ..data.SRR1552445.fastq.gz.subread.BAM
1                                      45
2                                       1
3                                       0
4                                      84
5                                     870
6                                       0
7                                       0
8                                       0
9                                       0
10                                      0
11                                      0
   ..data.SRR1552446.fastq.gz.subread.BAM
1                                      34
2                                       0
3                                       0
4                                      77
5                                     889
6                                       0
7                                       0
8                                       0
9                                       0
10                                      0
11                                      0
   ..data.SRR1552447.fastq.gz.subread.BAM
1                                      34
2                                       0
3                                       0
4                                      69
5                                     897
6                                       0
7                                       0
8                                       0
9                                       0
10                                      0
11                                      0
   ..data.SRR1552448.fastq.gz.subread.BAM
1                                      13
2                                       1
3                                       0
4                                      39
5                                     947
6                                       0
7                                       0
8                                       0
9                                       0
10                                      0
11                                      0
   ..data.SRR1552449.fastq.gz.subread.BAM
1                                      20
2                                       0
3                                       0
4                                      47
5                                     933
6                                       0
7                                       0
8                                       0
9                                       0
10                                      0
```

```
11                                      0
   ..data.SRR1552450.fastq.gz.subread.BAM
1                                      50
2                                       1
3                                       0
4                                      68
5                                     881
6                                       0
7                                       0
8                                       0
9                                       0
10                                      0
11                                      0
   ..data.SRR1552451.fastq.gz.subread.BAM
1                                      69
2                                       0
3                                       0
4                                      76
5                                     855
6                                       0
7                                       0
8                                       0
9                                       0
10                                      0
11                                      0
   ..data.SRR1552452.fastq.gz.subread.BAM
1                                      50
2                                       2
3                                       0
4                                      81
5                                     867
6                                       0
7                                       0
8                                       0
9                                       0
10                                      0
11                                      0
   ..data.SRR1552453.fastq.gz.subread.BAM
1                                      52
2                                       1
3                                       0
4                                      89
5                                     858
6                                       0
7                                       0
8                                       0
9                                       0
10                                      0
11                                      0
   ..data.SRR1552454.fastq.gz.subread.BAM
1                                      54
2                                       0
3                                       0
4                                      73
5                                     873
6                                       0
7                                       0
8                                       0
9                                       0
10                                      0
11                                      0
   ..data.SRR1552455.fastq.gz.subread.BAM
1                                      42
2                                       0
3                                       0
4                                      71
5                                     887
6                                       0
7                                       0
8                                       0
9                                       0
```

```
10                                  0
11                                  0
```

The counts for the samples are stored in fc$counts. Take a look at that.

```
## Take a look at the dimensions to see the number of genes
dim(fc$counts)
```

```
[1] 27179    12
```

```
## Take a look at the first 6 lines
head(fc$counts)
```

```
         ..data.SRR1552444.fastq.gz.subread.BAM
497097                                       0
100503874                                    0
100038431                                    0
19888                                        0
20671                                        0
27395                                        0
         ..data.SRR1552445.fastq.gz.subread.BAM
497097                                       0
100503874                                    0
100038431                                    0
19888                                        0
20671                                        0
27395                                        0
         ..data.SRR1552446.fastq.gz.subread.BAM
497097                                       0
100503874                                    0
100038431                                    0
19888                                        0
20671                                        0
27395                                        0
         ..data.SRR1552447.fastq.gz.subread.BAM
497097                                       0
100503874                                    0
100038431                                    0
19888                                        0
20671                                        0
27395                                        0
         ..data.SRR1552448.fastq.gz.subread.BAM
497097                                       0
100503874                                    0
100038431                                    0
19888                                        0
20671                                        0
27395                                        0
         ..data.SRR1552449.fastq.gz.subread.BAM
497097                                       0
100503874                                    0
100038431                                    0
19888                                        0
20671                                        0
27395                                        0
         ..data.SRR1552450.fastq.gz.subread.BAM
497097                                       0
100503874                                    0
100038431                                    0
19888                                        0
20671                                        0
27395                                        0
         ..data.SRR1552451.fastq.gz.subread.BAM
497097                                       0
100503874                                    0
100038431                                    0
19888                                        0
20671                                        0
27395                                        0
         ..data.SRR1552452.fastq.gz.subread.BAM
497097                                       0
100503874                                    0
100038431                                    0
19888                                        0
20671                                        0
27395                                        0
         ..data.SRR1552453.fastq.gz.subread.BAM
497097                                       0
100503874                                    0
100038431                                    0
19888                                        0
20671                                        0
27395                                        0
         ..data.SRR1552454.fastq.gz.subread.BAM
```

```
497097                                      0
100503874                                   0
100038431                                   0
19888                                       0
20671                                       0
27395                                       0
          ..data.SRR1552455.fastq.gz.subread.BAM
497097                                      0
100503874                                   0
100038431                                   0
19888                                       0
20671                                       0
27395                                       0
```

The row names of the fc$counts matrix represent the Entrez gene identifiers for each gene and the column names are the output filenames from calling the `align` function. The `annotation` slot shows the annotation information that `featureCounts` used to summarise reads over genes.

```
head(fc$annotation)
```

```
     GeneID                          Chr
1    497097               chr1;chr1;chr1
2 100503874                    chr1;chr1
3 100038431                         chr1
4     19888 chr1;chr1;chr1;chr1;chr1;chr1
5     20671      chr1;chr1;chr1;chr1;chr1
6     27395      chr1;chr1;chr1;chr1;chr1
                                      Start
1                       3214482;3421702;3670552
2                               3647309;3658847
3                                     3680155
4 4290846;4343507;4351910;4352202;4360200;4409170
5         4490928;4493100;4493772;4495136;4496291
6         4773198;4777525;4782568;4783951;4785573
                                        End      Strand Length
1                       3216968;3421901;3671498       -;-;-   3634
2                               3650509;3658904         -;-   3259
3                                     3681788           +   1634
4 4293012;4350091;4352081;4352837;4360314;4409241 -;-;-;-;-;-   9747
5         4492668;4493466;4493863;4495942;4496413   -;-;-;-;-   3130
6         4776801;4777648;4782733;4784105;4785726   -;-;-;-;-   4203
```

Challenge

1. Redo the counting over the exons, rather than the genes (specify `useMetaFeatures = FALSE`). Use the bam files generated doing alignment reporting only unique reads, and call the `featureCounts` object `fc.exon`. Check the dimension of the counts slot to see how much larger it is.
2. Using your ".multi.bam" files, redo the counting over genes, allowing for multimapping reads (specify `countMultiMappingReads = TRUE`), calling the object `fc.multi`. Check the stats.

Notes

- If you are sequencing your own data, the sequencing facility will almost always provide fastq files.
- For publicly available sequence data from GEO/SRA, the files are usually in the Sequence Read Archive (SRA) format. Prior to read alignment, these files need to be converted into the FASTQ format using the fastq-dump utility from the SRA Toolkit. See http://www.ncbi.nlm.nih.gov/books/NBK158900 for how to download and use the SRA Toolkit.
- By default, alignment is performed with `unique=TRUE`. If a read can be aligned to two or more locations, *Rsubread* will attempt to select the best location using a number of criteria. Only reads that have a unique best location are reported as being aligned. Keeping this default is recommended, as it avoids spurious signal from non-uniquely mapped reads derived from, e.g., repeat regions.
- The Phred offset determines the encoding for the base-calling quality string in the FASTQ file. For the Illumina 1.8 format onwards, this encoding is set at +33. However, older formats may use a +64 encoding. Users should ensure that the correct encoding is specified during alignment. If unsure, one can examine the first several quality strings in the FASTQ file. A good rule of thumb is to check whether lower-case letters are present (+64 encoding) or absent (+33).
- `featureCounts` requires gene annotation specifying the genomic start and end position of each exon of each gene. *Rsubread* contains built-in gene annotation for mouse and human. For other species, users will need to read in a data frame in GTF format to

define the genes and exons. Users can also specify a custom annotation file in SAF format. See the Rsubread users guide for more information, or try `?featureCounts`, which has an example of what an SAF file should like like.

Package versions used

```
sessionInfo()
```

```
R version 3.3.1 (2016-06-21)
Platform: x86_64-apple-darwin13.4.0 (64-bit)
Running under: OS X 10.11.6 (El Capitan)

locale:
[1] en_AU.UTF-8

attached base packages:
[1] stats     graphics  grDevices utils     datasets  base

other attached packages:
[1] Rsubread_1.22.3 knitr_1.14

loaded via a namespace (and not attached):
[1] magrittr_1.5  formatR_1.4   tools_3.3.1   stringi_1.1.1 methods_3.3.1
[6] stringr_1.1.0 evaluate_0.9
```

References

Dobin, Alexander, Carrie A Davis, Felix Schlesinger, Jorg Drenkow, Chris Zaleski, Sonali Jha, Philippe Batut, Mark Chaisson, and Thomas R Gingeras. 2013. “STAR: ultrafast universal RNA-seq aligner.” *Bioinformatics (Oxford, England)* 29 (1): 15–21. doi:10.1093/bioinformatics/bts635 (https://doi.org/10.1093/bioinformatics/bts635).

Fu, Nai Yang, Anne C Rios, Bhupinder Pal, Rina Soetanto, Aaron T L Lun, Kevin Liu, Tamara Beck, et al. 2015. “EGF-mediated induction of Mcl-1 at the switch to lactation is essential for alveolar cell survival.” *Nature Cell Biology* 17 (4): 365–75. doi:10.1038/ncb3117 (https://doi.org/10.1038/ncb3117).

Langmead, Ben, and Steven L Salzberg. 2012. “Fast gapped-read alignment with Bowtie 2.” *Nature Methods* 9 (4). Nature Publishing Group, a division of Macmillan Publishers Limited. All Rights Reserved.: 357–9. doi:10.1038/nmeth.1923 (https://doi.org/10.1038/nmeth.1923).

Liao, Yang, Gordon K Smyth, and Wei Shi. 2013. “The Subread aligner: fast, accurate and scalable read mapping by seed-and-vote.” *Nucleic Acids Research* 41: 16 pages.

Lun, Aaron T L, Yunshun Chen, and Gordon K Smyth. 2016. “It’s DE-licious: A Recipe for Differential Expression Analyses of RNA-seq Experiments Using Quasi-Likelihood Methods in edgeR.” *Methods in Molecular Biology (Clifton, N.J.)* 1418 (January): 391–416. doi:10.1007/978-1-4939-3578-9_19 (https://doi.org/10.1007/978-1-4939-3578-9_19).

Trapnell, Cole, Lior Pachter, and Steven L Salzberg. 2009. “TopHat: discovering splice junctions with RNA-seq.” *Bioinformatics* 25 (9): 1105–11. doi:doi:10.1093/bioinformatics/btp120 (https://doi.org/doi:10.1093/bioinformatics/btp120).